

Lecture 01

Software Re-Engineering



Engr. Maqsood Khatoon

Department of Software Engineering
National University of Computer & Emerging Sciences
maqsood.khatoon@nu.edu.pk

Course Overview

Information, Objectives, and Code of Ethics

Consultation Hours: Thursday 11:00 AM to 12:00 PM

| CODE OF ETHICS



Punctuality

All students must be on time. Attendance is taken in the first 5-10 minutes.



Focus

Remain attentive. Avoid unauthorized use of mobile phones or laptops.



Respect

Respect faculty and staff through professional speech and positive actions.

INTRODUCTION & PREREQUISITES

Course Objectives

Understand and practice various software reengineering techniques to effectively maintain and modify legacy systems for modern environments.

Prerequisites

SE-3001: Software Construction and Development. Familiarity with lifecycle models and programming paradigms is essential.

EVALUATION COMPONENTS

Component	Weight (%)	Details
Assignments	10%	Individual or group tasks.
Project	10%	Group of max 3 members with demo.
Midterm	30%	Standard university exam.
Final Exam	50%	Comprehensive assessment.

Text Book

Primary:

- **Re-Engineering legacy software**, David Lorge Parnas, Chris Birchall, Safari Books, Shelter Island, NY, 2016.
- **Object-Oriented Reengineering Patterns**. Morgan Kaufmann, S.Demeyer, S.Ducasse, and O.Nierstrasz. 2002.

Reference

- **Application Software Reengineering**, Afsharalam, Tendai Patenga, Dorling Kindersley Pvt. Ltd, Pearson Education in South Asia, 2010.
- **Software Maintenance and Evolution: a Roadmap**, K.H. Bennett and V.T. Rajlich, The Future of Software Engineering, ACM Press 2000.

Course Learning Outcomes (CLOs)

A. Course Learning Outcomes (CLOs)					
No.	Course Learning Outcomes (CLO)	Domain	Taxonomy Level	PLO	Tools
01	Define the concepts and techniques of software reengineering. Demonstrate and utilize software re-engineering techniques to maintain and modify software systems.	Cognitive	2	2	A, M, P, F
02	Examine maintenance related problems associated with object oriented software systems. Investigate and design complex reengineering and reverse engineering problems	Cognitive	5	4	A, M, P, F
Tool: A = Assignment,, P = Project, M = Mid-term, F=Final (End-term)					

Class Composition



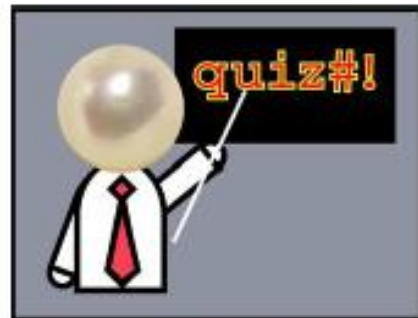
Lectures



Readings



Term Project



Quizzes



Assignments

Today's Agenda

- Introduction, Context, and Strategies
- State-of-the-art definitions
- Reengineering difficulties and challenges
- Reengineering Techniques
- Why Reengineering Projects Fail

Lecture 01

Introduction to Re-Engineering



WHAT IS REENGINEERING?

- ✓ Preparation or improvement to software, usually for increased maintainability, reusability or evolvability.

R. S. Arnold, *A Roadmap Guide to Software Reengineering Technology*, Software Reengineering, 1994.

- ✓ The examination and alteration of a subject system to reconstitute it in a new form and the subsequent implementation of the new form

E. Chikofsky and J. Cross, *Reverse Engineering and Design Recovery: A Taxonomy*, IEEE Software 7(1) Jan 1990: 13-17.



"We don't want to spend money on new computers, so we'll need you to keep the 10,000 old computers we have working."

WHAT IS REENGINEERING?

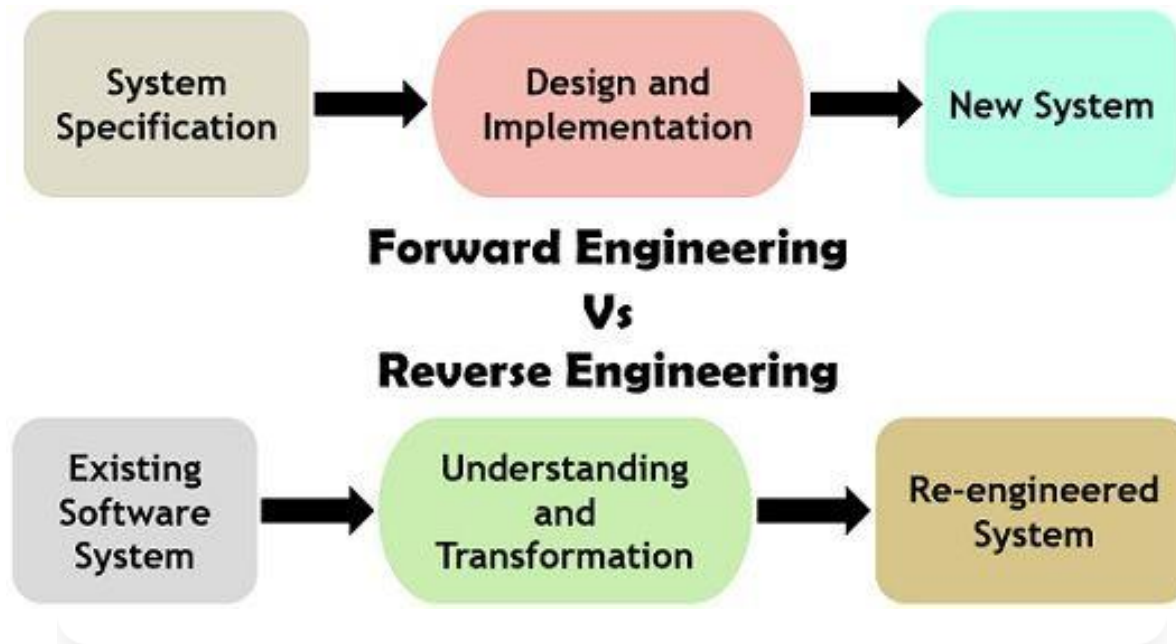
- ✓ Reengineering is the systematic transformation of an existing system into a new form to realize quality improvements in operation, system capability, functionality, performance, or evolvability at a lower cost, schedule, or risk to the customer.

S. Tilley and D. Smith, Perspectives on Legacy System Reengineering, SEI White Paper, 1995



"We don't want to spend money on new computers, so we'll need you to keep the 10,000 old computers we have working."

FORWARD VS REVERSE



Forward Engineering

Creating software by moving from high abstraction to low-level implementation.

Reverse Engineering

Analyzing software to identify components and relationships to represent at higher abstractions.

| RELATED TOPICS



Refactoring

Changing internal structure without altering external behavior.



Restructuring

Transformation at the same abstraction level to improve design.



Data Re-Engineering

Conversion of data formats without altering the content.

Business Process Re-Engineering

Concerned with the conversion of business process

| WHY REENGINEER?

- ✓ Emergence of new technologies.
 - ✓ Increasing market competition.
 - ✓ Cost-effective evolution option.
 - ✓ Quality of product importance.
 - ✓ Reusability of software products.
 - ✓ Regular maintenance needs.
 - ✓ Hardware obsolescence.
 - ✓ Preserving core functionality.
- 
-

| Reengineering Strategies

- ✓ Ignore – discard them
- ✓ Cold turkey – rewrite them from scratch
- ✓ Integrate – consolidate them into the current and future applications by access in place
- ✓ Gradual Migration – rearchitect and transition gradually
- ✓ Data Warehouse – build a “shadow” system to house the frequently accessed data

| Class Task 01

- ✓ Explore each of the Reengineering strategy mentioned above and bring real world examples of each.

| Factors Influencing Strategies

- ✓ Business value of the legacy systems – support current as well as future business processes.
- ✓ Flexibility and growth requirements – if legacy system does not need extensive changes for flexibility and growth then minimal effort for integration may be appropriate.
- ✓ Technical status of the legacy system – represents the quality of the system in terms of modularity, error rates, flexibility and utilization of current technologies.

| Categories of Legacy Applications

- ✓ Completely decomposable – all components are well structured and separable. Friendly to migration efforts
- ✓ Data decomposable – data services well structured and well-defined interfaces. Semi-friendly to migration efforts
- ✓ Program decomposable – program modules are well structured and have well-defined interfaces. Semi-friendly to migration efforts

| Reengineering difficulties and challenges

- ✓ Short life expectancy – not anticipated to last decades when first developed.
- ✓ Failure of process models and software engineering culture to treat evolution as a first-class activity – future requirements ignored.
- ✓ Satisfying constraints that existed at the time of development – hardware (memory and processing power).

Reengineering Issues

- ✓ Ease of change
how difficult will it be to change the legacy system
- ✓ Comprehensibility
how difficult is it to obtain an understanding of the systems that will need to be reengineered
- ✓ Size
what is the size of the systems that have to be reengineering
Decomposability
how easy or difficult is it to decompose the software system
- ✓ Degree of coupling
how coupled are the existing components
- ✓ Degree of cohesion
how cohesive are the components

| Reengineering Issues

- ✓ Granularity
can the system be reused in large chunks or does it need to be decomposed
 - ✓ Complexity
what is the level of complexity of the software
 - ✓ Degree of documentation
what documentation is available
 - ✓ Degree of abstraction
what abstractions are available
 - ✓ Language type & choice
what language(s) is the existing system written in and what will be used in the new system
 - ✓ Referential transparency
do the components always produce the same output
-

| Class Task 02

- ✓ Specify reasons why might a reengineered project fail ?